

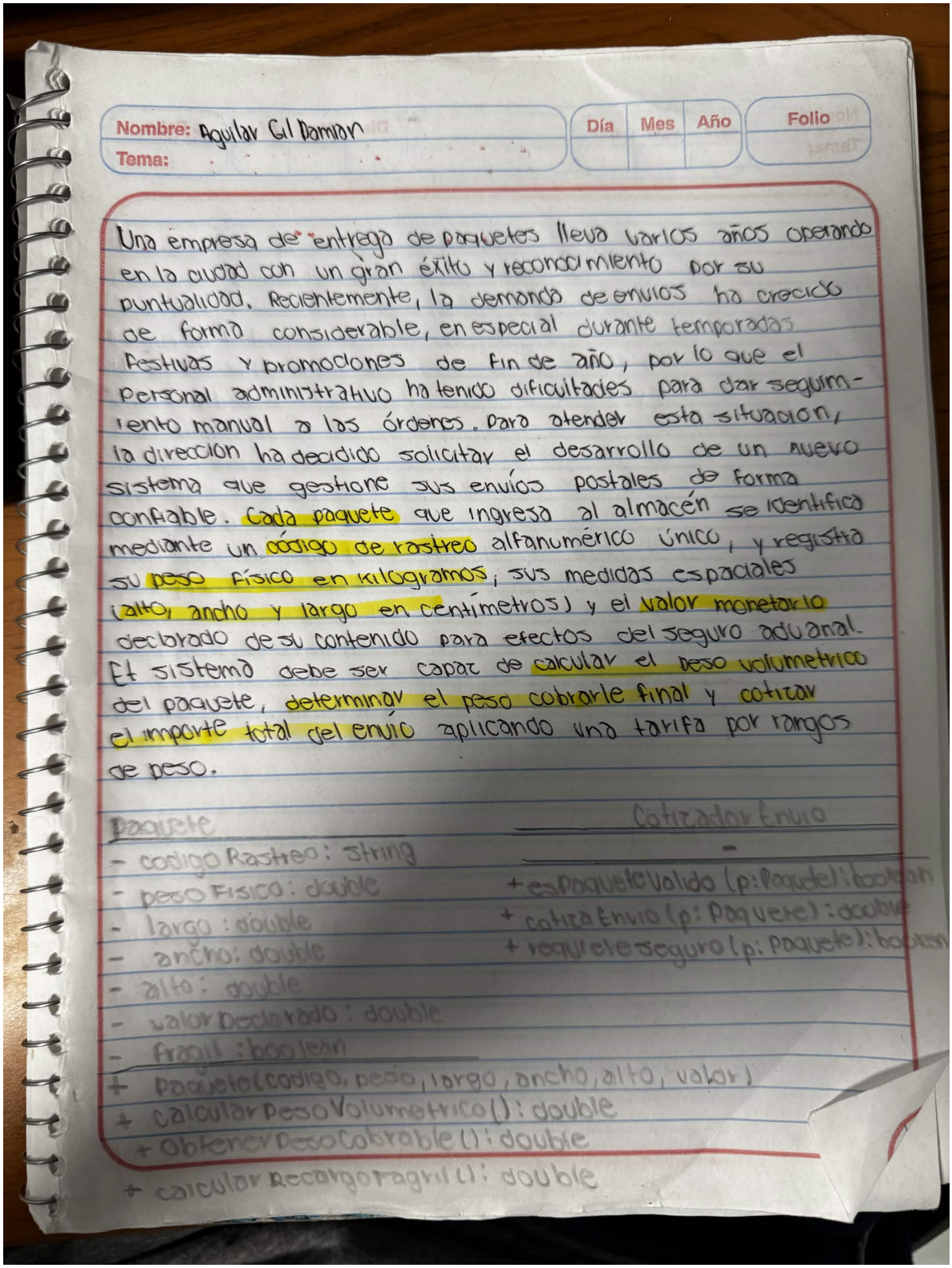
Universidad Nacional Autónoma De México

Facultad De Ciencias 2027 - 1

Profesor: Oscar Hernández Constantino
Ayudante de teoría: Geovani Martínez Martínez
Ayudante de lab: Víctor Manuel Mendiola Montes

Damian Pedro Aguilar Gil
324058075

Parte 1: Análisis gramatical y modelado



Nombre: Aguilar Gil Damian
Tema:
Día | Mes | Año | Folio

Una empresa de entrega de paquetes lleva varios años operando en la ciudad con un gran éxito y reconocimiento por su puntualidad. Recientemente, la demanda de envíos ha crecido de forma considerable, en especial durante temporadas festivas y promociones de fin de año, por lo que el personal administrativo ha tenido dificultades para dar seguimiento manual a las órdenes. Para atender esta situación, la dirección ha decidido solicitar el desarrollo de un nuevo sistema que gestione sus envíos postales de forma confiable. Cada paquete que ingresa al almacén se identifica mediante un código de rastreo alfanumérico único, y registra su peso físico en kilogramos, sus medidas espaciales (alto, ancho y largo en centímetros) y el valor monetario declarado de su contenido para efectos del seguro aduanal. El sistema debe ser capaz de calcular el peso volumétrico del paquete, determinar el peso cobrable final y cotizar el importe total del envío aplicando una tarifa por rangos de peso.

Paquete
- codigoRastreo: String
- pesoFisico: double
- largo: double
- ancho: double
- alto: double
- valorDeclarado: double
- fragil: boolean
+ Paquete(codigo, peso, largo, ancho, alto, valor)
+ calcularPesoVolumetrico(): double
+ ObtenerPesoCobrable(): double
+ calcularRecargoFragil(): double

CotizadorEnvio
+ esPaqueteValido(p: Paquete): boolean
+ cotizaEnvio(p: Paquete): double
+ requiereSeguro(p: Paquete): boolean

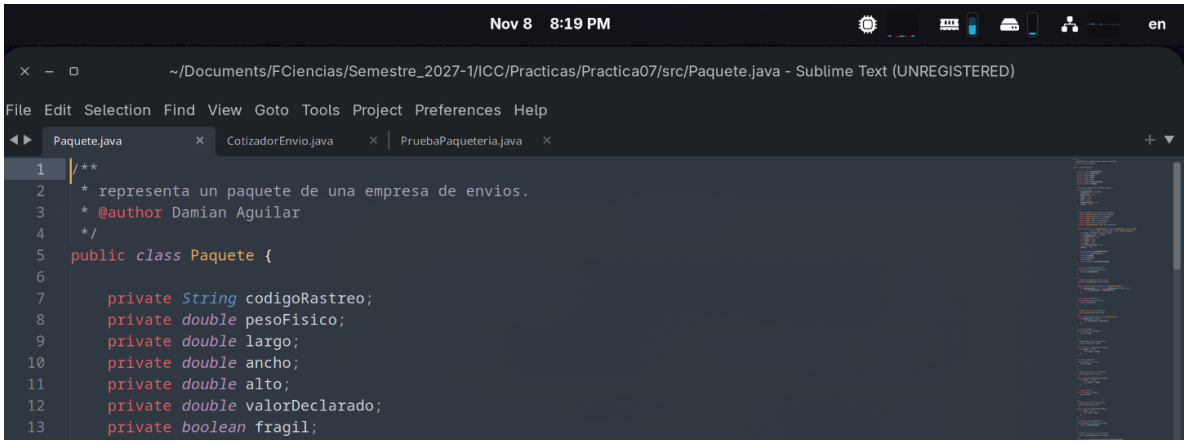
Primero leí el enunciado de la empresa de paquetería e identifiqué las palabras que podían servir para diseñar las clases del programa subrayé los elementos que consideré importantes principalmente los datos que tiene cada paquete y las acciones que debe realizar el sistema para calcular el costo de un envío. La clase principal que identifiqué fue Paquete porque representa el objeto del que necesitamos guardar información como su peso, dimensiones, código de rastreo y valor declarado y los métodos calculaPesoVolumetrico() y obtenerPesoCobrable() porque el enunciado menciona que debemos calcular el peso volumétrico y determinar cuál será el peso utilizado para cobrar el envío. Para los tipos de datos elegí string en el código de rastreo porque contiene letras y números para el peso, las dimensiones y el valor declarado utilicé double porque son cantidades que pueden tener decimales tmb declaré los atributos como privados para evitar que sus valores se modifiquen directamente desde otras clases.

La clase CotizadorEnvio que no tiene atributos propios porque su función es trabajar con los datos de los paquetes en ella coloqué los métodos esPaqueteValido() cotizarEnvio() y requiereSeguro(), los cuales permiten comprobar que un paquete sea válido, calcular el precio del envío y determinar si necesita un seguro obligatorio.

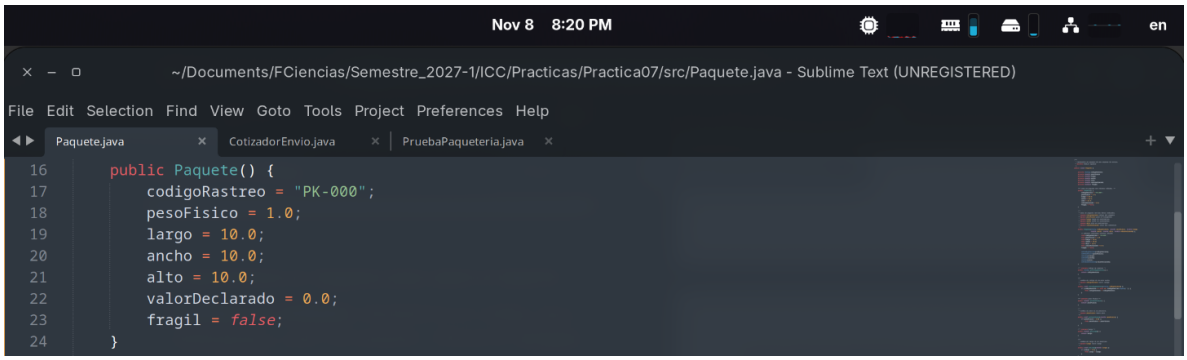
Yo cree el atributo **fragil** de tipo boolean ya que solamente necesito saber si el paquete es frágil o no igual añadí el método **calcularRecargoFragil()** hice esta función porque se me hace facil usarla y puede ser útil para una empresa de paquetería.

Y no use algunas cosas del enunciado como los años de operación de la empresa, la ciudad donde trabaja, su reconocimiento por puntualidad, las temporadas festivas, las promociones y las dificultades del personal administrativo ya que estos datos ayudan a entender el problema pero no son necesarios para representar los paquetes ni calcular los envíos.

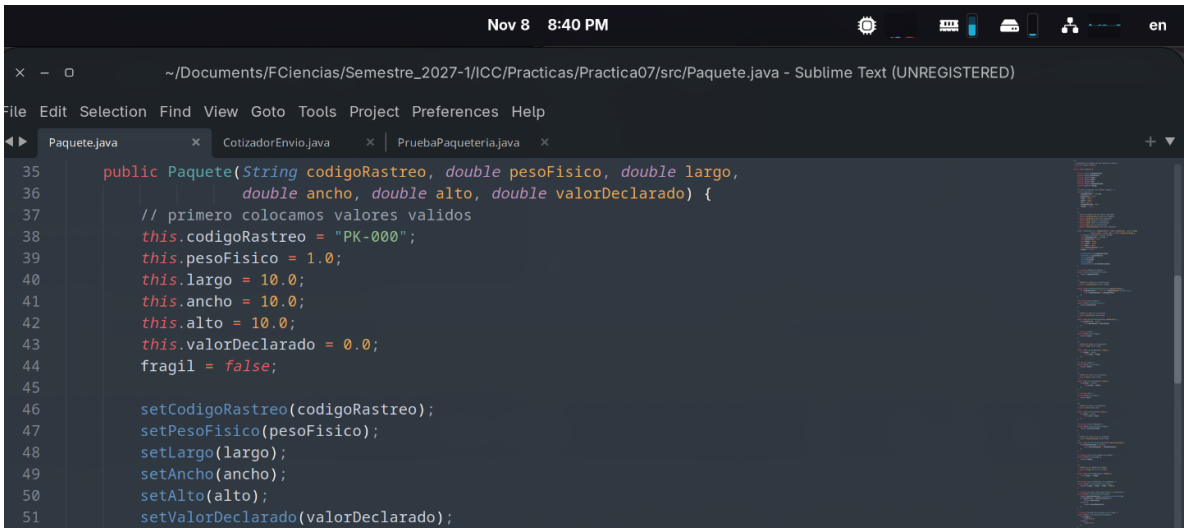
Parte 2: Implementación de la clase Paquete



Declare el código peso, dimensiones y valor del paquete utilize private para protegerlos.



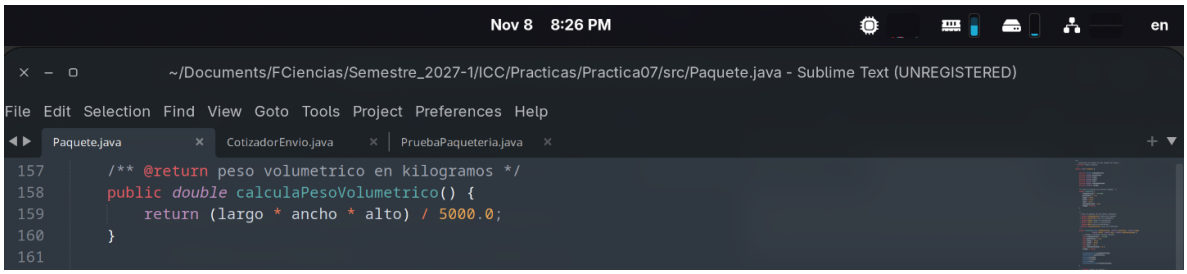
Cree el constructor por defecto inicialice un paquete con medidas y peso positivos.



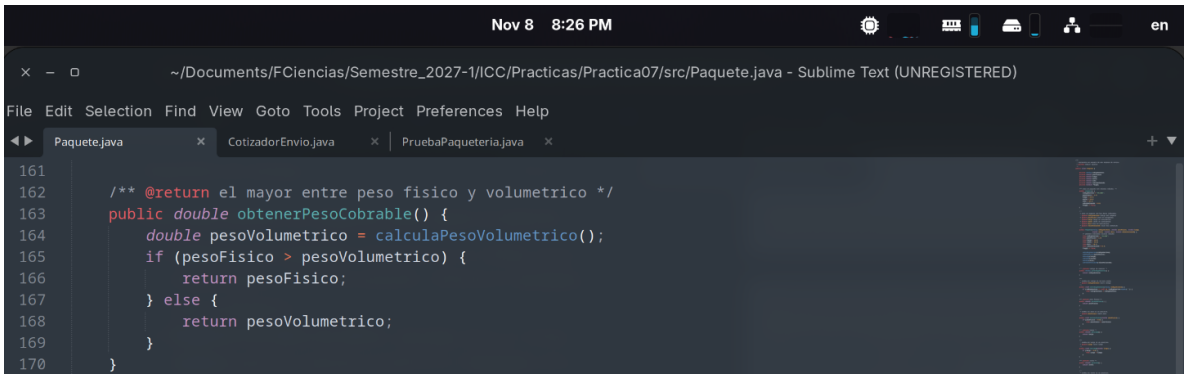
puse el constructor con parámetros este permite crear paquetes con los datos que nosotros pongamos.



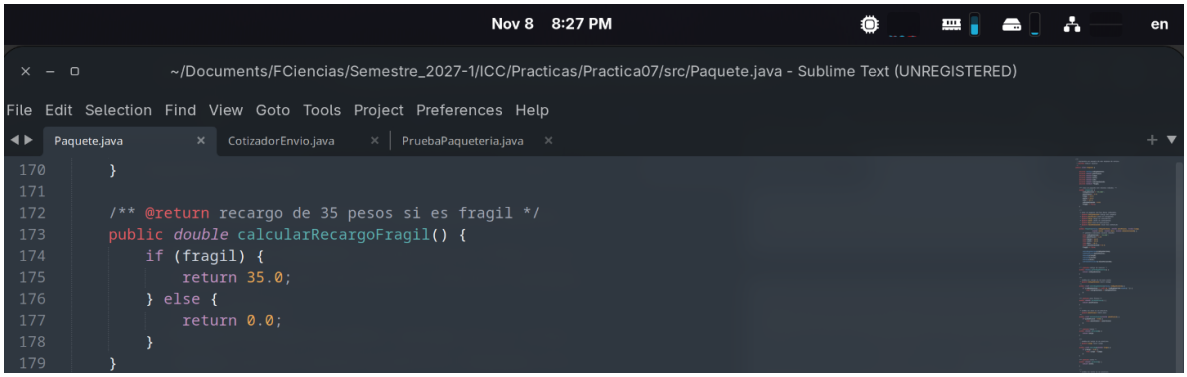
Para validar los datos use condiciones if para evitar medidas negativas o iguales a cero, pesos inválidos y valores declarados negativo de que si se recibe un dato inválido se conserva el valor válido anterior o el predeterminado. También agregue los getters y setters.



Calculamos el peso volumétrico multiplicamos largo, ancho y alto y dividimos el resultado entre 5000.



Para el peso cobrable compare el peso físico con el volumétrico y devolvemos el mayor.

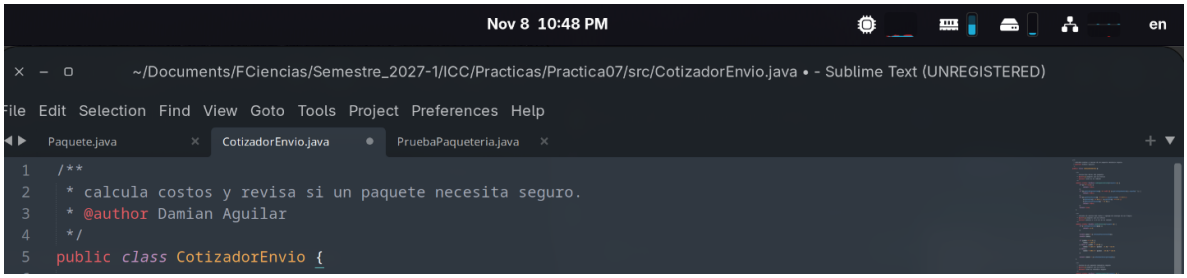


Acá pongo mi propuesta creae fragil y calcularRecargoFragil(), que devuelve $35 si el paquete es frágil y $0 en caso contrario.

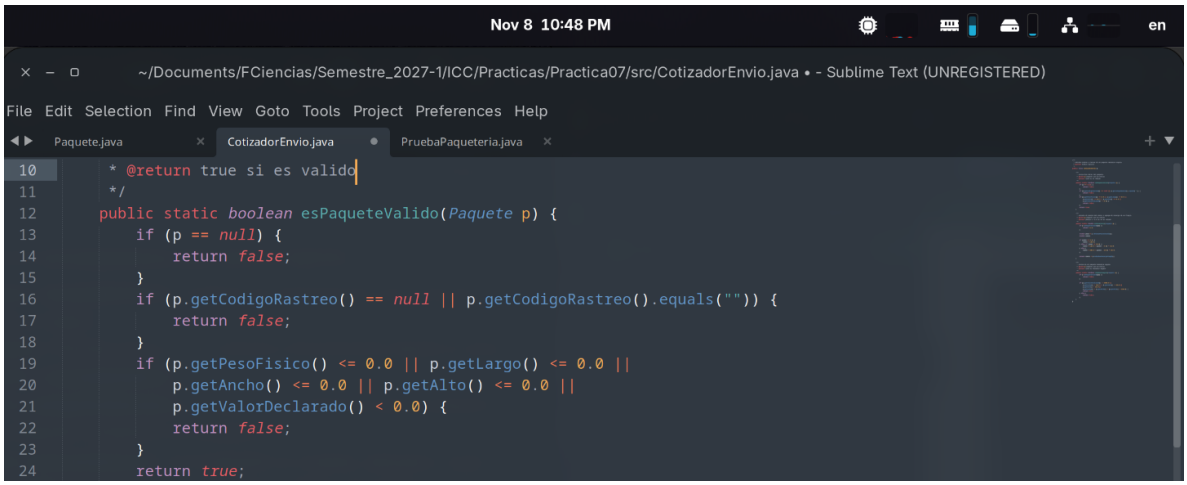
Decidí agregar esta función porque los paquetes frágiles necesitan mayor cuidado durante su transporte use boolean porque solamente necesito distinguir si el paquete es frágil o no y elegí un recargo fijo de $35 para que el cálculo fuera sencillo y se pudiera agregar directamente al precio del envío.

¿Qué validaciones decidiste incluir en el constructor y por qué? Decidí comprobar que el peso y las dimensiones fueran mayores a cero pq un paquete no debería tener medidas negativas o iguales a cero igual le revisé que el código de rastreo no estuviera vacío y que el valor declarado no fuera negativo para evitar que el objeto guarde datos incorrectos utilicé valores predeterminados cuando se reciben datos inválidos.

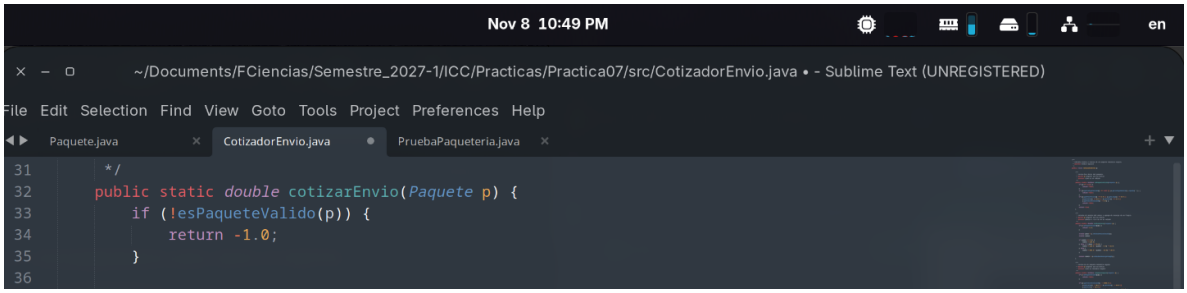
Parte 3: Implementación de la clase CotizadorEnvio



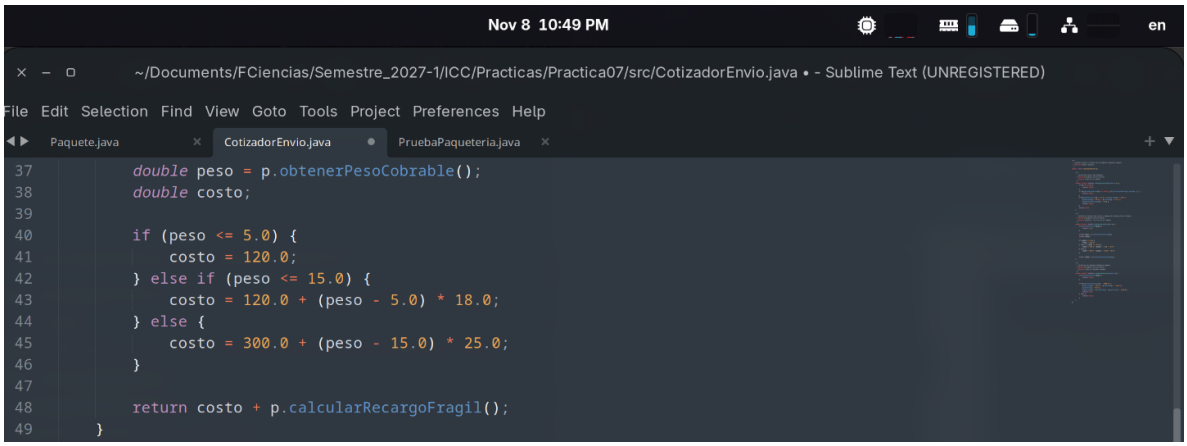
Creo la clase CotizadorEnvio q contiene las operaciones para calcular el precio y verificar el seguro.



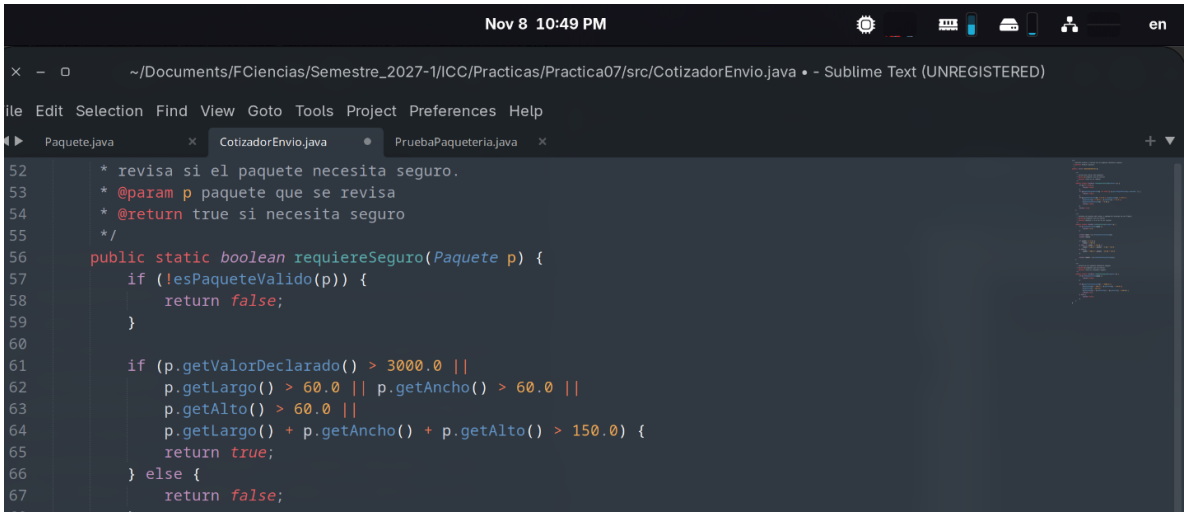
Utilizo static que me permite llamar a sus métodos directamente desde la clase sin crear un objeto, pongo esPaqueteValido() primero compruebo que el paquete no sea null y después revisamos sus atributos.



Acá cotizarEnvio() checa si el paquete es inválido e devuelve -1.0.



Agrego las tarifas con if, else if y else para aplicar los tres rangos de peso. En la linea 48 se suma el recargo por fragilidad al costo del envío.



pongo requiereSeguro() que comprueba si el valor declarado supera los $3,000 si alguna dimensión es mayor a 60 cm o si la suma de dimensiones supera 150 cm.

¿Por qué resulta conveniente que CotizadorEnvio tenga métodos estáticos en lugar de requerir crear instancias con new?

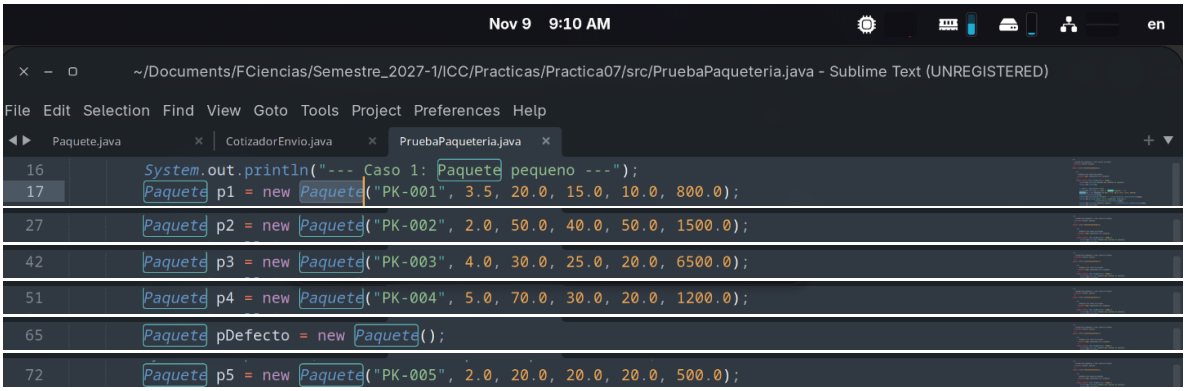
Es conveniente static porque no ocupa guardar información propia nadamas recibe los datos de un paquete y realiza operaciones con ellos de esta manera puedo llamar directamente a sus métodos sin tener que crear un objeto de CotizadorEnvio.

¿Qué diferencia conceptual y práctica existe entre una variable de referencia con valor null y un objeto instanciado con su constructor por defecto?

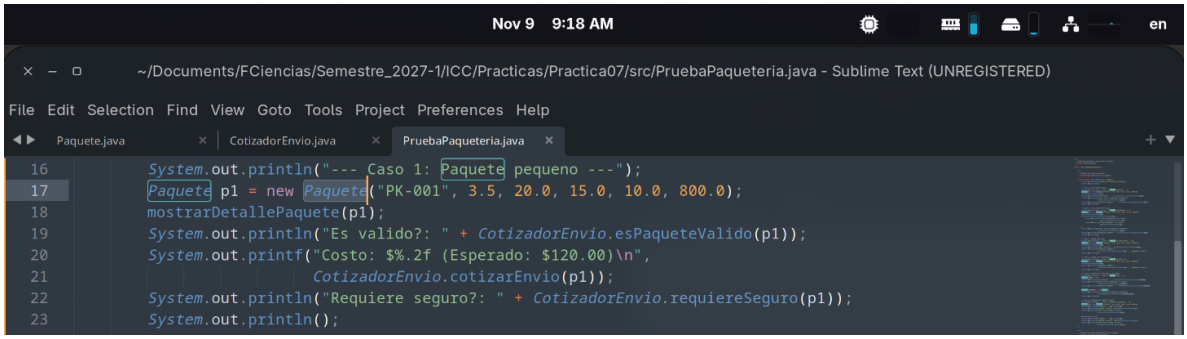
La diferencia es que una variable con valor null no apunta a ningún objeto por lo que no puedo consultar directamente sus atributos o métodos pero cuando ocupo new Paquete() sí creo un objeto que tiene ya tiene valores en su constructor y puedo trabajar normalmente con él.

Parte 4: Pruebas y verificación

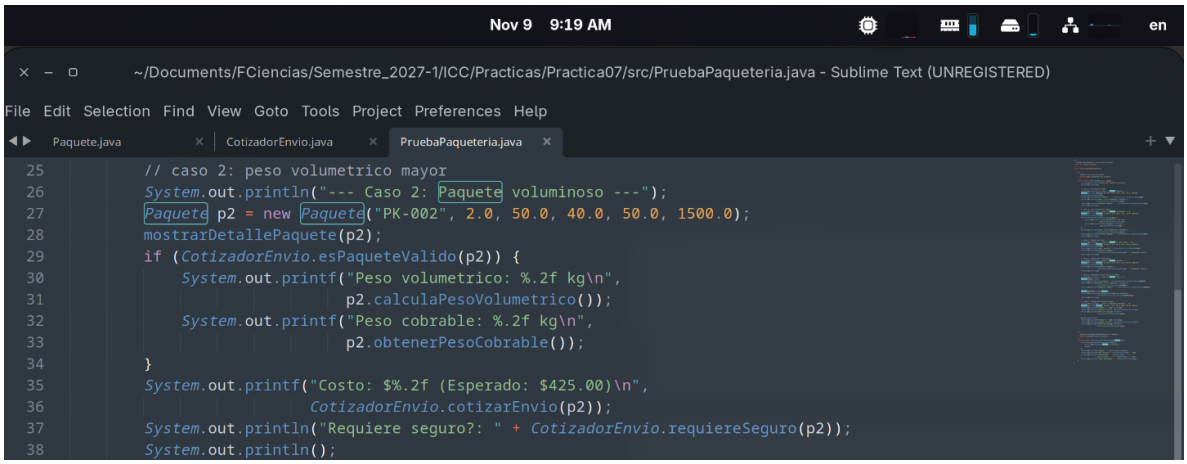
Creo los paquetes uso new Paquete() y el constructor con parámetros para generar los objetos.



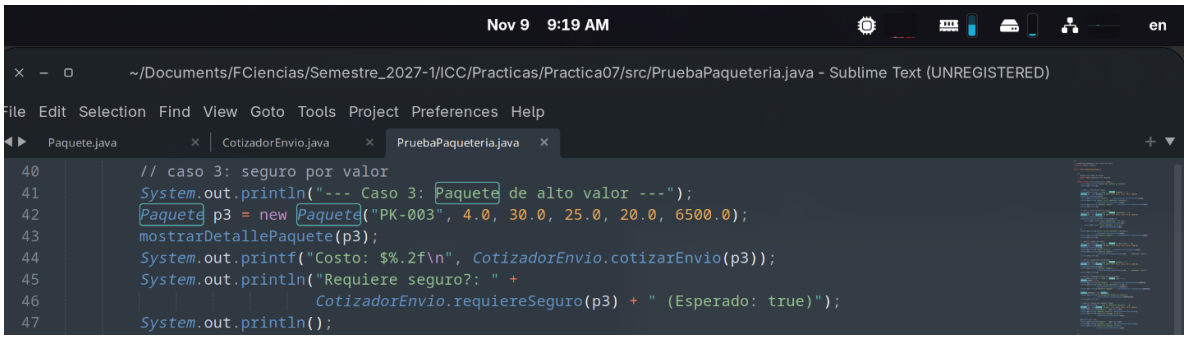
Probamos el caso 1 creo un paquete pequeño cuyo peso físico es mayor que el volumétrico. Esperábamos un costo de $120.



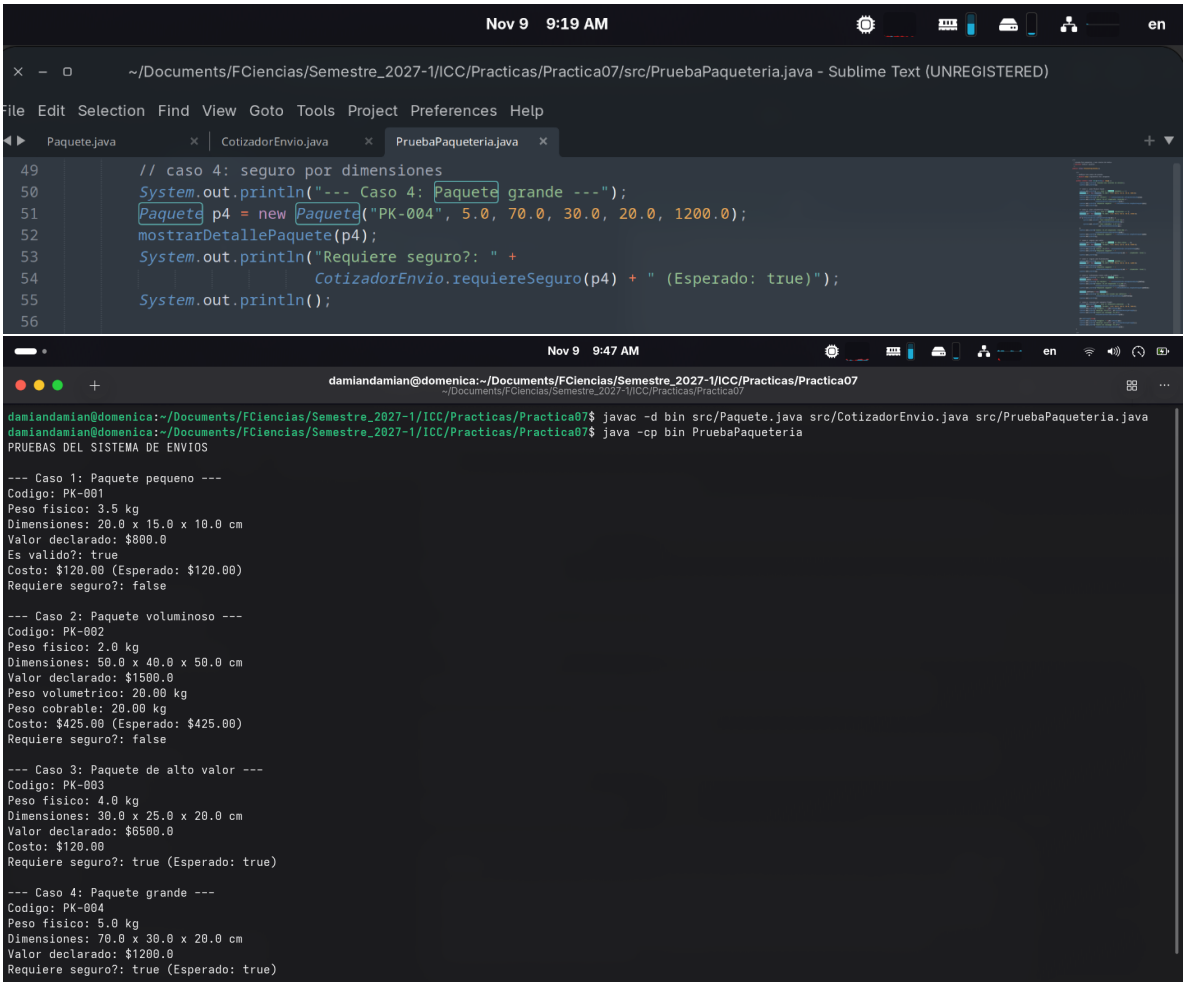
Para el caso 2 pongo un paquete gordo para comprobar que el programa utiliza el peso volumétrico. Esperábamos un costo de $425.



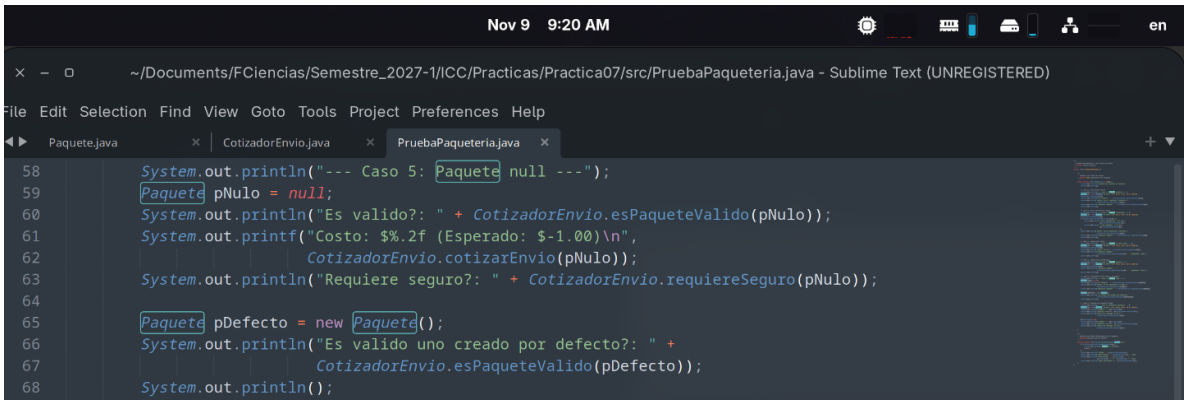
El caso 3 utilizo un paquete con valor declarado de $6,500 para comprobar que necesita seguro.



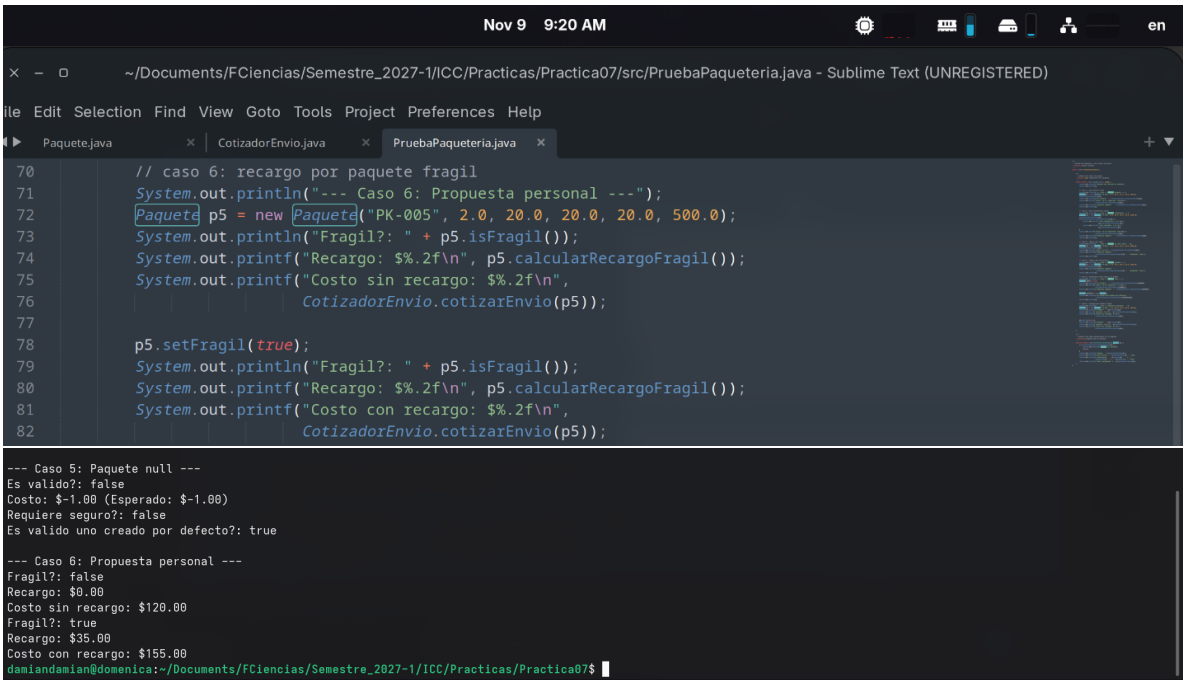
Caso 4 uso un paquete cuya dimensión mayor es de 70 cm para verificar que también requiere seguro.



En el caso 5 declaro una referencia null y verificamos que no es válida igual comprobamos que un objeto creado mediante el constructor por defecto sí es válido.



Para el caso 6 hice un paquete normal consultamos su precio y después lo marcamos como frágil mediante setFragil(true). Igual Comprobamos el recargo verificando que el costo aumenta de $120 a $155.



Explica qué casos de prueba diseñaste para validar tu propuesta personal, qué datos elegiste y qué comportamiento esperabas verificar.

Para comprobar mi propuesta creé un paquete con un peso de 2 kg, dimensiones de 20 × 20 × 20 cm y un valor declarado de $500, primero lo dejé como un paquete normal y comprobé que su envío costara $120, después utilicé setFragil(true) para marcarlo como frágil y volví a calcular el precio, esperaba que aumentara $35 y obtuve un total de $155, con esto pude comprobar que mi método funcionaba correctamente.

Conclusión.

En esta práctica aprendí a trabajar mejor con las clases y objetos en java principalmente a utilizar constructores, getters, setters y métodos para realizar operaciones con los atributos de un objeto, al principio me costó muchísimo entender cómo se relacionaban las clases Paquete y CotizadorEnvio pero con las pruebas pude comprender mejor cómo se utilizan sus métodos. Me gustó poder agregar una función propia y comprobar que funcionaba correctamente me sentí listo jaja.